MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

Name of faculty: Dazy Arya

Subject & Subject Code: CF&P,2FY2-08

Semester: II **Session: 2021-22 (Even Sem.)**

Branch: ~~CS~~,AI&DS, ME, CE, EE **Batch: C,D**

UNIT: V **Date of Submission: 28/03/2023**

Lecture No.	Topics Covered	Total Page
1	Function in C:Function -Basic,User defined functions	4
2	,inter function Communication(Call by value,call by reference)	7
3	standard functions, storage classes-auto, register, static, extern, scope rules	3
4	passing array to function,	7
5	passing structure to function,	3
6	introduction to recursion,recursive function,	6
7	pointer to functions	6
8	strings:Concepts,C strings ,string input/output function	8
9	,string manipulation functions,strings/data conversion.	6
10	input and output :Concepts of a file ,streams,text ,file input/output functions(standard library input/output function for files).	7

References:

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

28/3/2023
Dean Academics

faculty

HoD

LECTURE -01 (UNIT-05)

Syllabus :- Function in C

Function

Functions in C are the basic building blocks of a C program. A function is a set of statements enclosed within curly brackets ({}) that take inputs, do the computation, and provide the resultant output. You can call a function multiple times, thereby allowing reusability and modularity in C programming. It means that instead of writing the same code again and again for different arguments, you can simply enclose the code and make it a function and then call it multiple times by merely passing the various arguments.

Why Do We Need Functions in C Programming?

We need functions in C programming and even in other programming languages due to the numerous advantages they provide to the developer. Some of the key benefits of using functions are:

- Enables reusability and reduces redundancy
- Makes a code modular
- Provides abstraction functionality
- The program becomes easy to understand and manage
- Breaks an extensive program into smaller and simpler pieces

Basic Syntax of Functions

The basic syntax of functions in C programming is:

return_type function_name(arg1, arg2, … argn){

Body of the function //Statements to be processed

}

In the above syntax:

- return_type: Here, we declare the data type of the value returned by functions. However, not all functions return a value. In such cases, the keyword void indicates to the compiler that the function will not return any value.
- function_name: This is the function's name that helps the compiler identify it whenever we call it.
- arg1, arg2, ...argn: It is the argument or parameter list that contains all the parameters to be passed into the function. The list defines the data type, sequence, and the number of parameters to be passed to the function. A function may or may not require parameters. Hence, the parameter list is optional.
- Body: The function's body contains all the statements to be processed and executed whenever the function is called.

Note: The function name and parameters list are together known as the signature of a function in C programming.

General Aspects of Functions in C Programming

Functions in C programming have three general aspects: declaration, defining, and calling. Let's understand what these aspects mean.

Function Declaration

The function declaration lets the compiler know the name, number of parameters, data types of parameters, and return type of a function. However, writing parameter names during declaration is optional, as you can do that even while defining the function.

Function Call

As the name gives out, a function call is calling a function to be executed by the compiler. You can call the function at any point in the entire program. The only thing to take care of is that you need to pass as many arguments of the same data type as mentioned while declaring the function. If the function parameter does not differ, the compiler will execute the program and give the return value.

Function Definition

It is defining the actual statements that the compiler will execute upon calling the function. You can think of it as the body of the function. Function definition must return only one value at the end of the execution.

Here's an example with all the three general aspects of a function.

```
#include <stdio.h>

// Function  declaration

int max_Num(int i, int j){

   // Function definition

   if (i > j)

     return i;

   else

     return j;

}

// The main function. We will discuss about it later
```

```
int main(void){

    int x = 15, y = 20;

    // Calling the function to find the greater number among the two

    int m = max_Num(x, y);

    printf("The bigger number is %d", m);

    return 0;

}
```

Output: The bigger number is 20.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -02 (UNIT-05)

User Defined Function: User-defined functions in C language are defined by the programmer to perform specific operations. They are also known as "*tailor-made functions*" which are built only to satisfy the given condition. We can classify functions into call by value or call by reference.

Advantages of using functions in the program are:

- One can avoid duplication of code in the programs by using functions. Code can be written more quickly and be more readable as a result.
- Code can be divided and conquered using functions. This process is known as Divide and Conquer. It is difficult to write large amounts of code within the main function, as well as testing and debugging. Our one task can be divided into several smaller sub-tasks by using functions, thus reducing the overall complexity.
- For example, when using pow, sqrt, etc. in C without knowing how it is implemented, one can hide implementation details with functions.
- With little to no modifications, functions developed in one program can be used in another, reducing the development time.

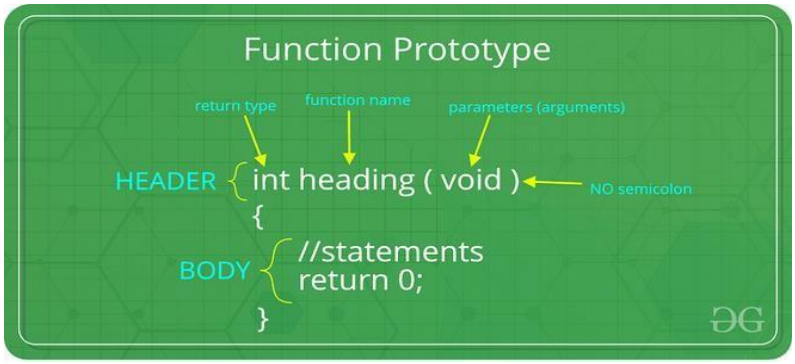
Elements of User-Defined Function in C

Function Prototype

A function prototype is also known as a function declaration which specifies the function's name, function parameters, and return type. The function prototype does not contain the body of the function. It is basically used to inform the compiler about the user-defined function which can be used in the later part of the program.

Syntax:

return_type function_name(type1 arg1, type2 arg2, … typeN argN);



Function Declaration

Example:

int sum(int x, int y);

Here return_type of the function is an integer and the name of the function is sum() which has two arguments of type int passed to the function.

Function Definition

Once the function has been called, the function definition contains the actual block of code that will be executed. The four components of a function definition are as follows:

- The return type of the function
- Function Name
- Function parameters
- Body of function

Function Calling

In order to transfer control to a user-defined function, we need to call it. Functions are called using their names followed by round brackets. Their arguments are passed inside the brackets. There are two types of function calls i.e, Call by value and Call by reference.

Syntax:

function_name(arg1, arg2, ... argN);

Example:

add(10,14);

A. Call by value

In call by value, a copy of the value is passed to the function and changes that are made to the function are not reflected back to the values. Actual and formal arguments are created in different memory locations.

Example:

```
// C program to show use of
// call by value
#include <stdio.h>
void swap(int a, int b)
{
  int temp = a;
  a = b;
  b = temp;
```

```
}

// Driver code

int main()

{

  int x = 10, y = 20;

  printf("Values of x and y before swap are: %d, %d\n", x, y);

  swap(x, y);

  printf("Values of x and y after swap are: %d, %d", x, y);

  return 0;

}
```

Output

```
Values of x and y before swap are: 10, 20
Values of x and y after swap are: 10, 20
```

Note:

Values aren't changed in the call by value since they aren't passed by reference.

B. Call by Reference

In a call by Reference, the address of the argument is passed to the function, and changes that are made to the function are reflected back to the values.

Example:

```
// C program to implement
```

```
// Call by Reference

#include <stdio.h>

void swap(int* a, int* b)

{

  int temp = *a;

  *a = *b;

  *b = temp;

}

// Driver code

int main()

{

  int x = 10, y = 20;

  printf("Values of x and y before swap are: %d, %d\n", x, y);

  swap(&x, &y);

  printf("Values of x and y after swap are: %d, %d", x, y);

  return 0;

}
```

Output

Values of x and y before swap are: 10, 20
Values of x and y after swap are: 20, 10

Return Statement

Return Statement is used to return value from the function. It basically terminates the execution of the function and the control of the program is returned to its caller. After the function call, the execution starts immediately. The return data type must match the defined data type in the function declaration and definition.

Syntax:

return (Expression)

Example:

return x+y;

Below is the program to demonstrate a user-defined function sum that returns the sum of arguments passed to it.

```
// C Program to show user-defined function
#include <stdio.h>
// Function declaration
int sum(int, int);
// Function definition
int sum(int x, int y)
{
  return x + y;
}
```

```
// Driver code

int main()

{

  int x = 10, y = 11;

  // Function call

  printf("Sum is %d ", sum(x, y));

  return 0;

}
```

Output

Sum is 21

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.

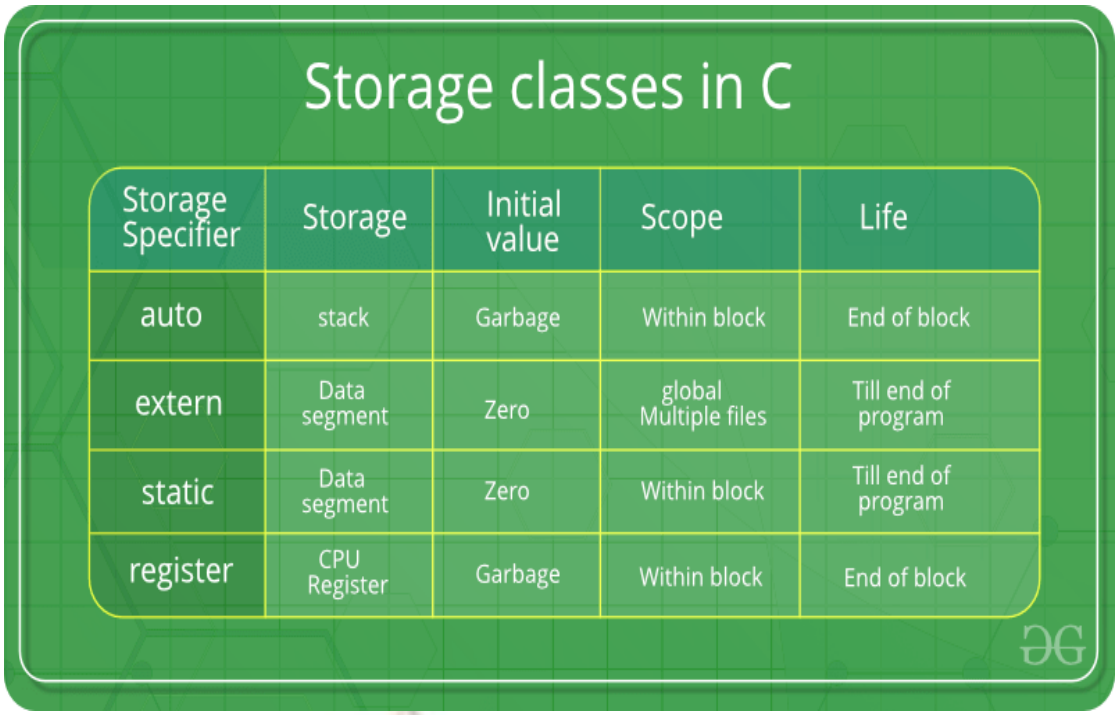
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -03 (UNIT-05)

STORAGE CLASSES IN C

Storage Classes are used to describe the features of a variable/function. These features basically include the scope, visibility and life-time which help us to trace the existence of a particular variable during the runtime of a program.

C language uses 4 storage classes, namely:



auto: This is the default storage class for all the variables declared inside a function or a block. Hence, the keyword auto is rarely used while writing programs in C language. Auto variables can be only accessed within the block/function they have been declared and not outside them (which defines their scope). Of course, these can be accessed within nested blocks within the parent block/function in which the auto variable was declared. However, they can be accessed outside their scope as well using the concept of pointers given here by pointing to the very exact memory location where the variables reside. They are assigned a garbage value by default whenever they are declared.

1. extern: Extern storage class simply tells us that the variable is defined elsewhere and not within the same block where it is used. Basically, the value is assigned to it in a different block and this can be overwritten/changed in a different block as well. So an extern variable is nothing but a global variable initialized with a legal value where it is declared in order to be used elsewhere. It can be accessed within any function/block. Also, a normal global variable can be made extern as well by placing the 'extern' keyword before its declaration/definition in any function/block. This basically signifies that we are not initializing a new variable but instead we are

using/accessing the global variable only. The main purpose of using extern variables is that they can be accessed between two different files which are part of a large program. For more information on how extern variables work, have a look at this link.

2. static: This storage class is used to declare static variables which are popularly used while writing programs in C language. Static variables have the property of preserving their value even after they are out of their scope! Hence, static variables preserve the value of their last use in their scope. So we can say that they are initialized only once and exist till the termination of the program. Thus, no new memory is allocated because they are not re-declared. Their scope is local to the function to which they were defined. Global static variables can be accessed anywhere in the program. By default, they are assigned the value 0 by the compiler.

3. register: This storage class declares register variables that have the same functionality as that of the auto variables. The only difference is that the compiler tries to store these variables in the register of the microprocessor if a free registration is available. This makes the use of register variables to be much faster than that of the variables stored in the memory during the runtime of the program. If a free registration is not available, these are then stored in the memory only. Usually few variables which are to be accessed very frequently in a program are declared with the register keyword which improves the running time of the program. An important and interesting point to be noted here is that we cannot obtain the address of a register variable using pointers.

4. storage_class var_data_type var_name;
5. Functions follow the same syntax as given above for variables. Have a look at the following C example for further clarification:

```
// A C program to demonstrate different storage
// classes
#include <stdio.h>
// declaring the variable which is to be made extern
// an initial value can also be initialized to x
int x;
void autoStorageClass()
{
    printf("\nDemonstrating auto class\n\n");
     // declaring an auto variable (simply
    // writing "int a=32;" works as well)
    auto int a = 32;
     // printing the auto variable 'a'
    printf("Value of the variable 'a'"
           " declared as auto: %d\n",
           a);
    printf("--------------------------------------------");
}
void registerStorageClass()
{
     printf("\nDemonstrating register class\n\n");
     // declaring a register variable
    register char b = 'G';
     // printing the register variable 'b'
    printf("Value of the variable 'b'"
           " declared as register: %d\n",
           b);
     printf("--------------------------------------------");
}
```

```
 void externStorageClass()
{
    printf("\nDemonstrating extern class\n\n");
    // telling the compiler that the variable
   // x is an extern variable and has been
   // defined elsewhere (above the main
   // function)
   extern int x;
    // printing the extern variables 'x'
   printf("Value of the variable 'x'"
         " declared as extern: %d\n",
         x);
    // value of extern variable x modified
   x = 2;
    // printing the modified values of
   // extern variables 'x'
   printf("Modified value of the variable 'x'"
         " declared as extern: %d\n",
         x);
    printf("..........");
}
 void staticStorageClass()
{
   int i = 0;
    printf("\nDemonstrating static class\n\n");
    // using a static variable 'y'
   printf("Declaring 'y' as static inside the loop.\n"
         "But this declaration will occur only"
         " once as 'y' is static.\n"
```

```
        "If not, then every time the value of 'y' "
        "will be the declared value 5"
        " as in the case of variable 'p'\n");
    printf("\nLoop started:\n");
    for (i = 1; i < 5; i++) {
      // Declaring the static variable 'y'
      static int y = 5;
      // Declare a non-static variable 'p'
      int p = 10;
      // Incrementing the value of y and p by 1
      y++;
      p++;
      // printing value of y at each iteration
      printf("\nThe value of 'y', "
          "declared as static, in %d "
          "iteration is %d\n",
          y, i);
      // printing value of p at each iteration
      printf("The value of non-static variable 'p', "
          "in %d iteration is %d\n",
          p, i);
   }
    printf("\nLoop ended:\n");
    printf("..........");
}
 int main()
{
    printf("A program to demonstrate"
        " Storage Classes in C\n\n");
```

```
    // To demonstrate auto Storage Class
    autoStorageClass();
    // To demonstrate register Storage Class
    registerStorageClass();
    // To demonstrate extern Storage Class
    externStorageClass();
    // To demonstrate static Storage Class
    staticStorageClass();
    // exiting
    printf("\n\nStorage Classes demonstrated");
    return 0;
}
// This code is improved by RishabhPrabhu
```

```
Output
...ster: 71
---------------------------------
Demonstrating extern class
Value of the variable 'x' declared as extern: 0
Modified value of the variable 'x' declared as extern: 2
---------------------------------
Demonstrating static class
Declaring 'y' as static inside the loop.
But this declaration will occur only once as 'y' is static.
If not, then every time the value of 'y' will be the declared value 5as in the case of
variable 'p'
Loop started:
The value of 'y', declared as static, in 6 iteration is 1
The value of non-static variable 'p', in 11 iteration is 1
The value of 'y', declared as static, in 7 iteration is 2
```

The value of non-static variable 'p', in 11 iteration is 2

The value of 'y', declared as static, in 8 iteration is 3

The value of non-static variable 'p', in 11 iteration is 3

The value of 'y', declared as static, in 9 iteration is 4

The value of non-static variable 'p', in 11 iteration is 4

Loop ended:

Storage Classes demonstrated

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.

2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -04 (UNIT-05)

How Arrays are Passed to Functions in C

A whole array cannot be passed as an argument to a function in C++. You can, however, pass a pointer to an array without an index by specifying the array's name.

In C, when we pass an array to a function say fun(), it is always treated as a pointer by fun(). The below example demonstrates the same.

```
// CPP Program to demonstrate passing
// an array to a function is always treated
// as a pointer
#include <iostream>
using namespace std;

// Note that arr[] for fun is
// just a pointer even if square
// brackets are used
void fun(int arr[]) // SAME AS void fun(int *arr)
{
    unsigned int n = sizeof(arr) / sizeof(arr[0]);
    cout << "\nArray size inside fun() is " << n;
}
 // Driver Code
int main()
{
    int arr[] = { 1, 2, 3, 4, 5, 6, 7, 8 };
    unsigned int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Array size inside main() is " << n;
    fun(arr);
    return 0;
}
```

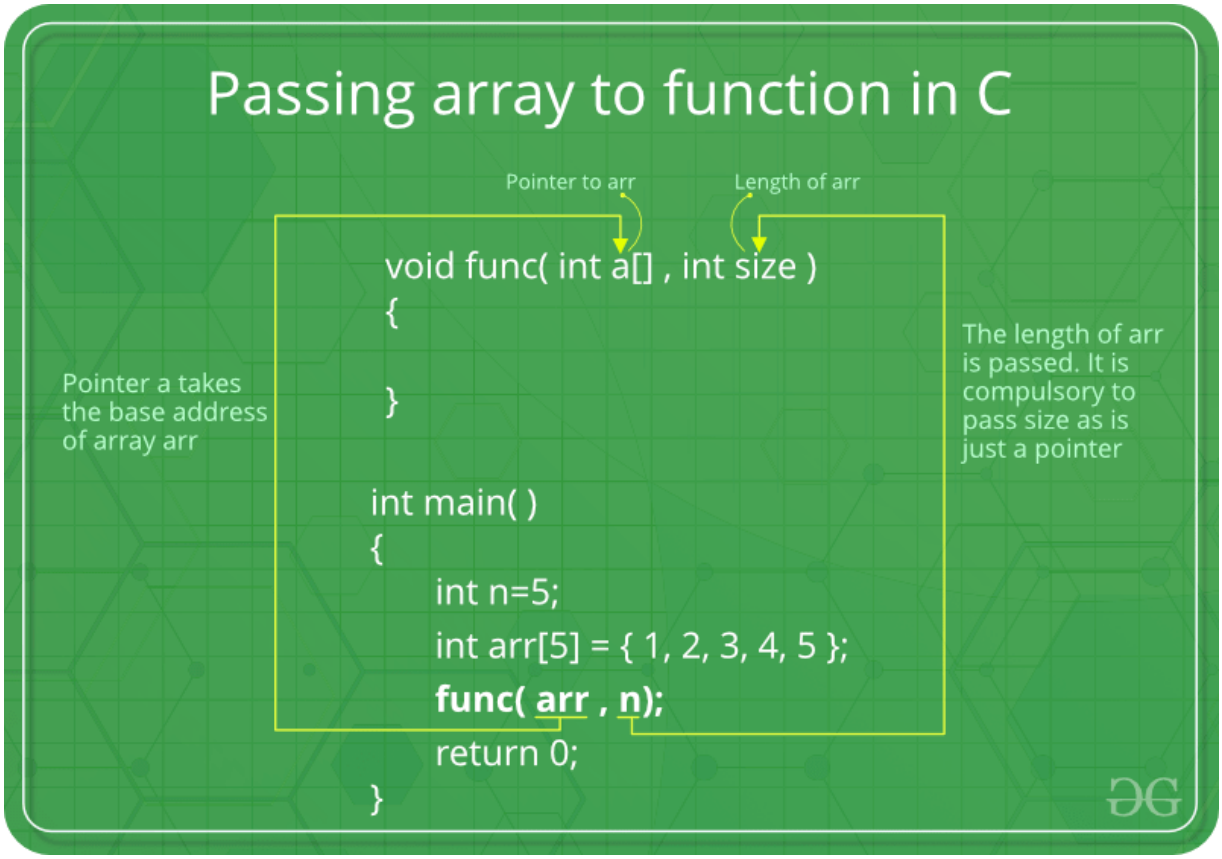
Output

```
Array size inside main() is 8
```

Array size inside fun() is 1

Therefore in C, we must pass the size of the array as a parameter. Size may not be needed only in the case of '\0' terminated character arrays, size can be determined by checking the end of string character.

Following is a simple example to show how arrays are typically passed in C:




```
#include <stdio.h>
 void fun(int *arr, unsigned int n)
{
   int i;
   for (i=0; i<n; i++)
     printf("%d  ", arr[i]);
```

```
}
 // Driver program
int main()
{
   int arr[] = {1, 2, 3, 4, 5, 6, 7, 8};
   unsigned int n = sizeof(arr)/sizeof(arr[0]);
   fun(arr, n);
   return 0;
}
```

Output

```
1  2  3  4  5  6  7  8
```

Exercise: Predict the output of the below C programs:

Program 1:

```
#include <stdio.h>
void fun(int arr[], unsigned int n)
{
int i;
for (i=0; i<n; i++)
    printf("%d ", arr[i]);
}

// Driver program
int main()
{
int arr[] = {1, 2, 3, 4, 5, 6, 7, 8};
unsigned int n = sizeof(arr)/sizeof(arr[0]);
```

```
fun(arr, n);
return 0;
}
```

Output

```
1 2 3 4 5 6 7 8
```

Program 2:

```
#include <stdio.h>
void fun(int *arr)
{
  int i;
  unsigned int n = sizeof(arr)/sizeof(arr[0]);
  for (i=0; i<n; i++)
    printf("%d  ", arr[i]);
}

// Driver program
int main()
{
  int arr[] = {1, 2, 3, 4, 5, 6, 7, 8};
  fun(arr);
  return 0;
}
```

Output

```
1 2
```

According to the processor, size of pointer changes for 32 bit computer it assigns 4 bytes to pointer then output becomes 1.

Program 3:

```
// Program 3
#include <stdio.h>
#include <string.h>

void fun(char *arr)
{
   int i;
   unsigned int n = strlen(arr);
   printf("n = %d\n", n);
   for (i=0; i<n; i++)
     printf("%c  ", arr[i]);
}
 // Driver program
int main()
{
   char arr[] = "geeksquiz";
   fun(arr);
   return 0;
}
```

Output

```
n = 9
g e e k s q u i z
#include <stdio.h>
#include <string.h>
```

```
 void fun(char *arr)
{
   int i;
   unsigned int n = strlen(arr);
   printf("n = %d\n", n);
   for (i=0; i<n; i++)
     printf("%c  ", arr[i]);
}
 // Driver program
int main()
{
   char arr[] = {'g', 'e', 'e', 'k', 's', 'q', 'u', 'i', 'z'};
   fun(arr);
   return 0;
}
```

Output

```
n = 11
g e e k s q u i z
```

NOTE: The character array in the above program is not '\0' terminated. (See this for details)

Now These were some of the common approaches that we use but do you know that there is a better way to do the same. For this, we first need to look at the drawbacks of all the above-suggested methods:

Drawbacks:

- A major drawback of the above method is compiler has no idea about what you are passing. What I mean here is for compiler we are just passing an int* and we know that this is pointing to the array but the compiler doesn't know this.

To verify my statement you can call for-each loop on your array. You will surely get an error saying no callable begin, end function found.

This is because the passing array is like actually passing an integer pointer and it itself has no information about the underlying array hence no iterator is provided.

A structure is a user-defined data type in C/C++. A structure creates a data type that can be used to group items of possibly different types into a single type.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -05 (UNIT-05)

How to pass structure as an argument to the functions?

Passing of structure to the function can be done in two ways:

- By passing all the elements to the function individually.
- By passing the entire structure to the function.

In this article, entire structure is passed to the function. This can be done using call by reference as well as call by value method.

C Program to Pass Structure Variable to Function

In below program, we are passing a member variable to printAge function and whole structure variable to printEmployeeDetails function.

```
#include <stdio.h>
 struct employee {
 char name[100];
 int age;
 float salary;
 char department[50];
};
 void printEmployeeDetails(struct employee emp){
   printf("Employee Details\n");
   printf(" Name : %s\n Age : %d\n Salary = %f\n Dept : %s\n",
       emp.name, emp.age, emp.salary, emp.department);
}
void printAge(int age){
```

```
    printf("\nAge = %d\n", age);
}
 int main(){
   struct employee employee_one, *ptr;

   printf("Enter Name, Age, Salary and Department of Employee\n");
   scanf("%s %d %f %s", &employee_one.name, &employee_one.age,
       &employee_one.salary, &employee_one.department);

   /* Passing structure variable to function */
   printEmployeeDetails(employee_one);

   /* Passing structure member to function */
   printAge(employee_one.age);

   return 0;
}
```

Program Output

```
Enter Name, Age, Salary and Department of Employee
Rose 35 1234.5 Purchase
Employee Details
 Name : Rose
 Age : 35
 Salary = 1234.500000
 Dept : Purchase

Age = 35
```

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.

2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -06 (UNIT-05)

WHAT IS RECURSION?

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function. Using a recursive algorithm, certain problems can be solved quite easily. Examples of such problems are Towers of Hanoi (TOH), Inorder/Preorder/Postorder Tree Traversals, DFS of Graph, etc. A recursive function solves a particular problem by calling a copy of itself and solving smaller subproblems of the original problems. Many more recursive calls can be generated as and when required. It is essential to know that we should provide a certain case in order to terminate this recursion process. So we can say that every time the function calls itself with a simpler version of the original problem.

Need of Recursion

Recursion is an amazing technique with the help of which we can reduce the length of our code and make it easier to read and write. It has certain advantages over the iteration technique which will be discussed later. A task that can be defined with its similar subtask, recursion is one of the best solutions for it. For example; The Factorial of a number.

Properties of Recursion:

- Performing the same operations multiple times with different inputs.
- In every step, we try smaller inputs to make the problem smaller.
- Base condition is needed to stop the recursion otherwise infinite loop will occur.

A Mathematical Interpretation

Let us consider a problem that a programmer has to determine the sum of first n natural numbers, there are several ways of doing that but the simplest approach is simply to add the numbers starting from 1 to n. So the function simply looks like this,

approach(1) – Simply adding one by one

f(n) = 1 + 2 + 3 + ...+ n

but there is another mathematical approach of representing this,

approach(2) – Recursive adding

f(n) = 1 n=1

f(n) = n + f(n-1) n>1

There is a simple difference between the approach (1) and approach(2) and that is in approach(2) the function " f( ) " itself is being called inside the function, so this phenomenon is named recursion, and the function containing recursion is called recursive function, at the end, this is a great tool in the hand of the programmers to code some problems in a lot easier and efficient way.

How are recursive functions stored in memory?

Recursion uses more memory, because the recursive function adds to the stack with each recursive call, and keeps the values there until the call is finished. The recursive function uses LIFO (LAST IN FIRST OUT) Structure just like the stack data structure.
https://www.geeksforgeeks.org/stack-data-structure/

What is the base condition in recursion?
In the recursive program, the solution to the base case is provided and the solution to the bigger problem is expressed in terms of smaller problems.

```
int fact(int n)
{
   if (n < = 1) // base case
      return 1;
```

```
    else
        return n*fact(n-1);
}
```

In the above example, the base case for n < = 1 is defined and the larger value of a number can be solved by converting to a smaller one till the base case is reached.

How a particular problem is solved using recursion?
The idea is to represent a problem in terms of one or more smaller problems, and add one or more base conditions that stop the recursion. For example, we compute factorial n if we know the factorial of (n-1). The base case for factorial would be n = 0. We return 1 when n = 0.

```
int fact(int n)
{
    // wrong base case (it may cause
    // stack overflow).
    if (n == 100)
        return 1;

    else
        return n*fact(n-1);
}
```

If fact(10) is called, it will call fact(9), fact(8), fact(7), and so on but the number will never reach 100. So, the base case is not reached. If the memory is exhausted by these functions on the stack, it will cause a stack overflow error.

Recursion VS Iteration

SR	Recursion	Iteration

No.		
1)	Terminates when the base case becomes true.	Terminates when the condition becomes false.
2)	Used with functions.	Used with loops.
3)	Every recursive call needs extra space in the stack memory.	Every iteration does not require any extra space.
4)	Smaller code size.	Larger code size.

Now, let's discuss a few practical problems which can be solved by using recursion and understand its basic working. For basic understanding please read the following articles.
Basic understanding of Recursion.
Problem 1: Write a program and recurrence relation to find the Factorial of n where n>2 .
Mathematical Equation:

1 if n == 0 or n == 1;
f(n) = n*f(n-1) if n> 1;

Recurrence Relation:

T(n) = 1 for n = 0
T(n) = 1 + T(n-1) for n > 0

Recursive Program:
Input: n = 5
Output:
factorial of 5 is: 120
Implementation:

```
// C code to implement factorial
#include <stdio.h>
 // Factorial function
int f(int n)
{
    // Stop condition
    if (n == 0 || n == 1)
        return 1;
     // Recursive condition
    else
        return n * f(n - 1);
}
 // Driver code
int main()
{
    int n = 5;
    printf("factorial of %d is: %d", n, f(n));
    return 0;
}
```

Output

factorial of 5 is: 120

Time complexity: O(n)

Auxiliary Space: O(n)

Summary of Recursion:

- There are two types of cases in recursion i.e. recursive case and a base case.
- The base case is used to terminate the recursive function when the case turns out to be true.
- Each recursive call makes a new copy of that method in the stack memory.
- Infinite recursion may lead to running out of stack memory.
- Examples of Recursive algorithms: Merge Sort, Quick Sort, Tower of Hanoi, Fibonacci Series, Factorial Problem, etc.
- In C, like normal data pointers (int *, char *, etc), we can have pointers to functions. Following is a simple example that shows declaration and function call using function pointer.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar, Let US C,BPB Publication.

LECTURE -07 (UNIT-05)

POINTER TO FUNCTIONS :

C Function Pointer

As we know that we can create a pointer of any data type such as int, char, float, we can also create a pointer pointing to a function. The code of a function always resides in memory, which means that the function has some address. We can get the address of memory by using the function pointer.

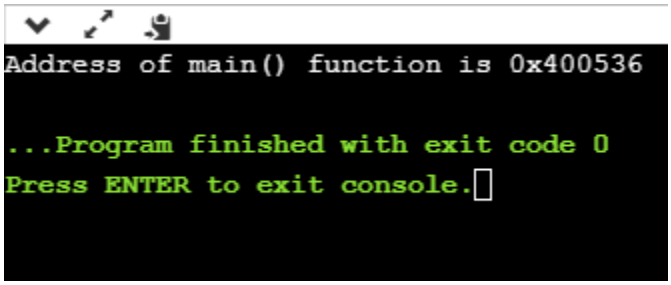
Let's see a simple example.

```
1.  #include <stdio.h>
2.  int main()
3. {
4.      printf("Address of main() function is %p",main);
5.      return 0;
6. }
```

The above code prints the address of **main()** function.

Output This is a modal window.

The media playback was aborted due to a corruption problem or because the media used features your browser did not support.



In the above output, we observe that the main() function has some address. Therefore, we conclude that every function has some address.

Declaration of a function pointer

Till now, we have seen that the functions have addresses, so we can create pointers that can contain these addresses, and hence can point them.

Syntax of function pointer

```
1. return type (*ptr_name)(type1, type2…);
```

For example:

```
1. int (*ip) (int);
```

In the above declaration, *ip is a pointer that points to a function which returns an int value and accepts an integer value as an argument.

```
1. float (*fp) (float);
```

In the above declaration, ***fp** is a pointer that points to a function that returns a float value and accepts a float value as an argument.

We can observe that the declaration of a function is similar to the declaration of a function pointer except that the pointer is preceded by a '*****'. So, in the above declaration, fp is declared as a function rather than a pointer.

Till now, we have learnt how to declare the function pointer. Our next step is to assign the address of a function to the function pointer.

```
1. float (*fp) (int , int);   // Declaration of a function pointer.
2. float func( int , int );    // Declaration of  function.
3. fp = func;                  // Assigning address of func to the fp pointer.
```

In the above declaration, '**fp**' pointer contains the address of the '**func**' function.

Note: Declaration of a function is necessary before assigning the address of a function to the function pointer.

Calling a function through a function pointer

We already know how to call a function in the usual way. Now, we will see how to call a function using a function pointer.

Suppose we declare a function as given below:

```
1. float func(int , int);     // Declaration of a function.
```

Calling an above function using a usual way is given below:

```
1. result = func(a , b);     // Calling a function using usual ways.
```

Calling a function using a function pointer is given below:

```
1. result = (*fp)( a , b);    // Calling a function using function pointer.
```

Or

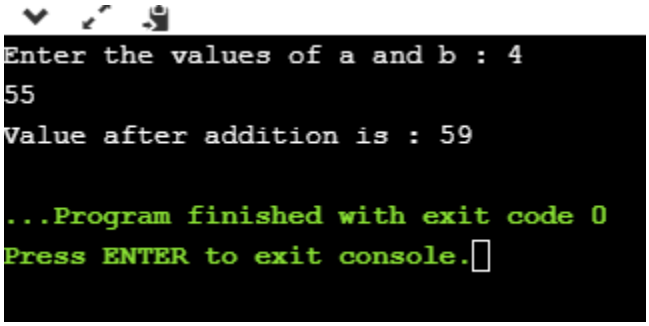
```
1. result = fp(a , b);        // Calling a function using function pointer, and indirection operator
   can be removed.
```

The effect of calling a function by its name or function pointer is the same. If we are using the function pointer, we can omit the indirection operator as we did in the second case. Still, we use the indirection operator as it makes it clear to the user that we are using a function pointer.

Let's understand the function pointer through an example.

```
1.  ..#include <stdio.h>
2.  In..t add(int,int);
3.  int .main()
4. {
5.      int a,b;
6.      int (*ip)(int,int);
7.      int result;
8.      printf("Enter the values of a and b : ");
9.      scanf("%d %d",&a,&b);
10.     ip=add;
11.     result=(*ip)(a,b);
12.     printf("Value after addition is : %d",result);
13.      return 0;
14. }
15. int add(int a,int b)
16. {
17.      int c=a+b;
18.      return c;
19. }
```

Output



Passing a function's address as an argument to other function

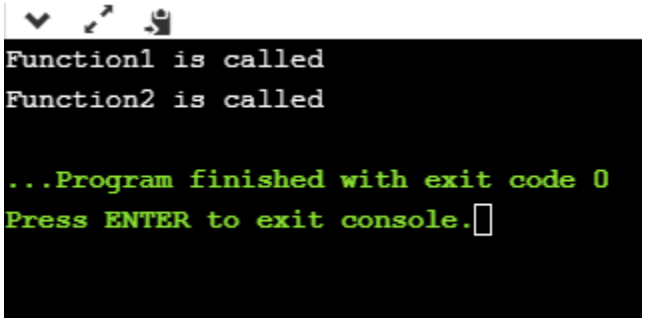
We can pass the function's address as an argument to other functions in the same way we send other arguments to the function.

Let's understand through an example.

```
1.  include <stdio.h>
2.  void func1(void (*ptr)());
3.  void func2();
4.  int main()
5. {
6.      func1(func2);
7.       return 0;
8. }
9.  void func1(void (*ptr)())
10. {
11.     printf("Function1 is called");
12.     (*ptr)();
13. }
14. void func2()
15. {
16.     printf("\nFunction2 is called");
17. }
```

In the above code, we have created two functions, i.e., func1() and func2(). The func1() function contains the function pointer as an argument. In the main() method, the func1() method is called in which we pass the address of func2. When func1() function is called, 'ptr' contains the address of 'func2'. Inside the func1() function, we call the func2() function by dereferencing the pointer 'ptr' as it contains the address of func2.

Output



REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

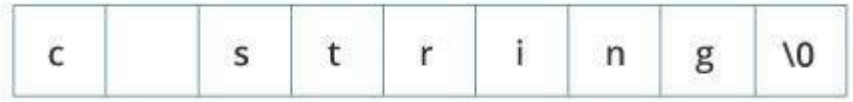
LECTURE -08 (UNIT-05)

STRINGS:

In C programming, a string is a sequence of characters terminated with a null character \0. For example:

char c[] = "c string";

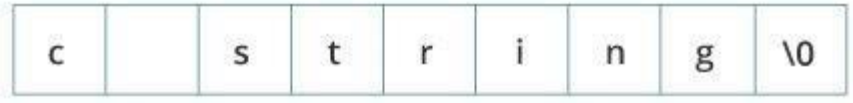
When the compiler encounters a sequence of characters enclosed in the double quotation marks, it appends a null character \0 at the end by default.



Memory DiagramIn C programming, a string is a sequence of characters terminated with a null character \0. For example:

char c[] = "c string";

When the compiler encounters a sequence of characters enclosed in the double quotation marks, it appends a null character \0 at the end by default.

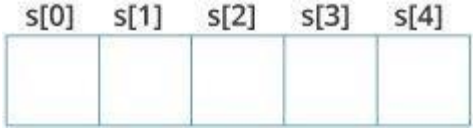


Memory Diagram

How to declare a string?

Here's how you can declare strings:

char s[5];



String Declaration in C

Here, we have declared a string of 5 characters.

How to initialize strings?

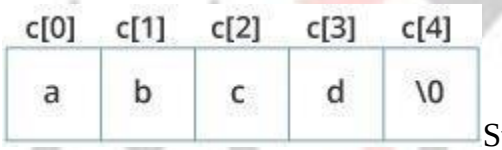
You can initialize strings in a number of ways.

char c[] = "abcd";

char c[50] = "abcd";

char c[] = {'a', 'b', 'c', 'd', '\0'};

char c[5] = {'a', 'b', 'c', 'd', '\0'};



String Initialization in C

Let's take another example:

char c[5] = "abcde";

Here, we are trying to assign 6 characters (the last character is '\0') to a char array having 5 characters. This is bad and you should never do this.

Assigning Values to Strings

Arrays and strings are second-class citizens in C; they do not support the assignment operator once it is declared. For example,

```
char c[100];
c = "C programming";  // Error! array type is not assignable.
```

Note: Use the strcpy() function to copy the string instead.

Read String from the user

You can use the scanf() function to read a string.

The scanf() function reads the sequence of characters until it encounters whitespace (space, newline, tab, etc.).

Example 1: scanf() to read a string

```
#include <stdio.h>
int main()
{
   char name[20];
   printf("Enter name: ");
   scanf("%s", name);
   printf("Your name is %s.", name);
   return 0;
}
```

Output

```
Enter name: Dennis Ritchie
Your name is Dennis.
```

Even though *Dennis Ritchie* was entered in the above program, only *"Dennis"* was stored in the *name* string. It's because there was a space after *Dennis*.

Also notice that we have used the code *name* instead of &name with scanf().

scanf("%s", name);

This is because *name* is a char array, and we know that array names decay to pointers in C.

Thus, the *name* in scanf() already points to the address of the first element in the string, which is why we don't need to use &.

How to read a line of text?

You can use the fgets() function to read a line of string. And, you can use puts() to display the string.

Example 2: gets() and puts()

```
#include <stdio.h>
int main()
{
   char name[30];
   printf("Enter name: ");
   gets(name; // read string
   printf("Name: ");
   puts(name);    // display string
   return 0;
}
```

Output

Enter name: Tom Hanks

Name: Tom Hanks

Here, we have used gets() function to read a string from the user.

Gets(name);

To print the string, we have used puts(name);.

Note: The gets() function can also be to take input from the user. However, it is removed from the C standard.

It's because gets() allows you to input any length of characters. Hence, there might be a buffer overflow.

Function	Work of Function
strlen()	computes string's length
strcpy()	copies a string to another
strcat()	concatenates(joins) two strings
strcmp()	compares two strings
strlwr()	converts string to lowercase
strupr()	converts string to uppercase

Strings handling functions are defined under "string.h" header file.

#include <string.h>

C strlen()

The strlen() function calculates the length of a given string.

The strlen() function takes a string as an argument and returns its length. The returned value is of type size_t (an unsigned integer type).

It is defined in the <string.h> header file.

Example: C strlen() function

```
#include <stdio.h>
#include <string.h>
int main()
{
   char a[20]="Program";
   char b[20]={'P','r','o','g','r','a','m','\0'};
   // using the %zu format specifier to print size_t
   printf("Length of string a = %zu \n",strlen(a));
   printf("Length of string b = %zu \n",strlen(b));
   return 0;
}
```

Run Code

Output

```
Length of string a = 7
Length of string b = 7
```

Note that the strlen() function doesn't count the null character \0 while calculating the length.

Calculate Length of String without Using strlen() Function

```
#include <stdio.h>
int main() {
   char s[] = "Programming is fun";
   int i;
   for (i = 0; s[i] != '\0'; ++i);
```

```
    printf("Length of the string: %d", i);
    return 0;
}
```

Output

Length of the string: 18

C strcpy()

In this tutorial, you will learn to use the strcpy() function in C programming to copy strings (with the help of an example).

C strcpy()

The function prototype of strcpy() is:

```
char* strcpy(char* destination, const char* source);
```

- The strcpy() function copies the string pointed by *source* (including the null character) to the destination.
- The strcpy() function also returns the copied string.

The strcpy() function is defined in the *string.h* header file.

Example: C strcpy()

```
#include <stdio.h>
#include <string.h>

I
int main() {
  char str1[20] = "C programming";
```

```
  char str2[20];

  // copying str1 to str2
  strcpy(str2, str1);

  puts(str2); // C programming

  return 0;
}
```

Run Code

Output

C programming

Note: When you use strcpy(), the size of the destination string should be large enough to store the copied string. Otherwise, it may result in undefined behavior.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -09 (UNIT-05)

Copy String Without Using strcpy()

```
#include <stdio.h>
int main() {
    char s1[100], s2[100], i;
    printf("Enter string s1: ");
    fgets(s1, sizeof(s1), stdin);

    for (i = 0; s1[i] != '\0'; ++i) {
        s2[i] = s1[i];
    }

    s2[i] = '\0';
    printf("String s2: %s", s2);
    return 0;
}
```

Run Code

Output

```
Enter string s1: Hey fellow programmer.
String s2: Hey fellow programmer.
```

C strcmp()

In this tutorial, you will learn to compare two strings using the strcmp() function.

The strcmp() compares two strings character by character. If the strings are equal, the function returns 0.

Return Value from strcmp()

Return Value	Remarks
0	if strings are equal
>0	if the first non-matching character in *str1* is greater (in ASCII) than that of *str2*.
<0	if the first non-matching character in *str1* is lower (in ASCII) than that of *str2*.

The strcmp() function is defined in the string.h header file.

```
#include <stdio.h>
#include<string.h>
int main()
{
  char str1[20];  // declaration of char array
  char str2[20];  // declaration of char array
  int value; // declaration of integer variable
  printf("Enter the first string : ");
  scanf("%s",str1);
  printf("Enter the second string : ");
  scanf("%s",str2);
  // comparing both the strings using strcmp() function
  value=strcmp(str1,str2);
  if(value==0)
  printf("strings are same");
```

- else
- printf("strings are not same");
- return 0;

}

```
#include <stdio.h>
#include <string.h>

int main()
{
        char Str1[100], Str2[100];
        int result, i;

        printf("\n Please Enter the First String :  ");
        gets(Str1);

        printf("\n Please Enter the Second String :  ");
        gets(Str2);

        for(i = 0; Str1[i] == Str2[i] && Str1[i] == '\0'; i++);

        if(Str1[i] < Str2[i])
        {
                printf("\n str1 is Less than str2");
        }
        else if(Str1[i] > Str2[i])
        {
                printf("\n str2 is Less than str1");
        }
```

```
        else
        {
                printf("\n str1 is Equal to str2");
        }

        return 0;
}
```

C strcat()

In C programming, the strcat() function contcatenates (joins) two strings.

The function definition of strcat() is:

```
char *strcat(char *destination, const char *source)
```

It is defined in the string.h header file.

---

strcat() arguments

As you can see, the strcat() function takes two arguments:

destination - destination string
source - source string

The strcat() function concatenates the destination string and the source string, and the result is stored in the destination string.

Example: C strcat() function

```
#include <stdio.h>
#include <string.h>
int main() {
  char str1[100] = "This is ", str2[] = "programiz.com";

  // concatenates str1 and str2
  // the resultant string is stored in str1.
  strcat(str1, str2);

  puts(str1);
  puts(str2);

  return 0;
}
```

Output

```
This is programiz.com
programiz.com
```

Note: When we use strcat(), the size of the destination string should be large enough to store the resultant string. If not, we will get the segmentation fault error.

Concatenate Two Strings Without Using strcat()

```
#include <stdio.h>
int main()
{
```

```
  char str1[50], str2[50], i, j;
  printf("\nEnter first string: ");
  scanf("%s",str1);
  printf("\nEnter second string: ");
  scanf("%s",str2);
  len=strlen(str1);
  for(i=len;j=0; str1[j]!='\0';i++,j++);

  /* This loop would concatenate the string str2 at
   * the end of str1
   */
    {
     str1[i]=str2[j];
  }
  // \0 represents end of string
  str1[i]='\0';
  printf("\nOutput: %s",str1);

  return 0;
}
```

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -10 (UNIT-05)

File Handling in C?

File handling refers to the method of storing data in the C program in the form of an output or input that might have been generated while running a C program in a data file, i.e., a binary file or a text file for future analysis and reference in that very program.

What is a File in C?

A file refers to a source in which a program stores the information/data in the form of bytes of sequence on a disk (permanently). The content available on a file isn't volatile like the compiler memory in C. But the program can perform various operations, such as creating, opening, reading a file, or even manipulating the data present inside the file. This process is known as file handling in C.

Why Do We Need File Handling in C?

There are times when the output generated out of a program after its compilation and running do not serve our intended purpose. In such cases, we might want to check the program's output various times. Now, compiling and running the very same program multiple times becomes a tedious task for any programmer. It is exactly where file handling becomes useful.

Let us look at a few reasons why file handling makes programming easier for all:

- Reusability: File handling allows us to preserve the information/data generated after we run the program.
- Saves Time: Some programs might require a large amount of input from their users. In such cases, file handling allows you to easily access a part of a code using individual commands.
- Commendable storage capacity: When storing data in files, you can leave behind the worry of storing all the info in bulk in any program.

- Portability: The contents available in any file can be transferred to another one without any data loss in the computer system. This saves a lot of effort and minimises the risk of flawed coding.

Types of Files in a C Program

When referring to file handling, we refer to files in the form of data files. Now, these data files are available in 2 distinct forms in the C language, namely:

- Text Files
- Binary Files

Text Files

The text files are the most basic/simplest types of files that a user can create in a C program. We create the text files using an extension *.txt* with the help of a simple text editor. In general, we can use notepads for the creation of .txt files. These files store info internally in ASCII character format, but when we open these files, the content/text opens in a human-readable form.

Text files are, thus, very easy to access as well as use. But there's one major disadvantage; it lacks security. Since a .txt file can be accessed easily, information isn't very secure in it. Added to this, text files consume a very large space in storage.

To solve these problems, we have a different type of file in C programs, known as binary files.

Binary Files

The binary files store info and data in the binary format of 0's and 1's (the binary number system). Thus, the files occupy comparatively lesser space in the storage. In simpler words, the binary files store data and info the same way a computer holds the info in its memory. Thus, it can be accessed very easily as compared to a text file.

The binary files are created with the extension .bin in a program, and it overcomes the drawback of the text files in a program since humans can't read it; only machines can. Thus, the information becomes much more secure. Thus, binary files are safest in terms of storing data files in a C program.

Operators/Functions that We Use for File Handling in C

We can use a variety of functions in order to open a file, read it, write more data, create a new file, close or delete a file, search for a file, etc. These are known as file handling operators in C.

Here's a list of functions that allow you to do so:

Description of Function	Function in Use
used to open an existing file or a new file	fopen()
writing data into an available file	fprintf()
reading the data available in a file	fscanf()
writing any character into the program file	fputc()
reading the character from an available file	fgetc()
used to close the program file	fclose()
used to set the file pointer to the intended file position	fseek()
writing an integer into an available file	fputw()
used to read an integer from the given file	fgetw()
used for reading the current position of a file	ftell()

sets an intended file pointer to the file's beginning itself rewind()

Note: It is important to know that we must declare a file-type pointer when we are working with various files in a program. This helps establish direct communication between a program and the files.

Here is how you can do it:

FILE *fpointer;

Out of all the operations/functions mentioned above, let us discuss some of the basic operations that we perform in the C language.

Operations Done in File Handling

The process of file handling enables a user to update, create, open, read, write, and ultimately delete the file/content in the file that exists on the C program's local file system. Here are the primary operations that you can perform on a file in a C program:

- Opening a file that already exists
- Creating a new file
- Reading content/ data from the existing file
- Writing more data into the file
- Deleting the data in the file or the file altogether

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -11(UNIT-05)

Opening a File in the Program – to create and edit data

We open a file with the help of the fopen() function that is defined in the header file- stdio.h.

Here is the syntax that we follow when opening a file:

ptr = fopen (“openfile” , “openingmode”);

Let us take a look at an example for the same,

fopen (“E:\\myprogram\\recentprogram.txt” , “w”);

fopen (“E:\\myprogram\\previousprogram.bin” , “rb”);

- Here, if we suppose that the file – recentprogram.txt doesn’t really exist in the E:\\myprogram location. Here, we have used the mode “w”. Thus, the first function will create a new file with the name recentprogram.txt and then open it for writing (since we have used the “w” mode).
- The “w” here refers to writing mode. It allows a programmer to overwrite/edit and create the contents in a program file.
- Now, let us take a look at the second binary previousprogram.bin file that is present in the E:\\myprogram location. Thus, the second function here will open the file (that already exists) for reading in the “rb” binary mode.
- The “rb” refers to the reading mode. It only allows you to read a file, but not overwrite it. Thus, it will only read this available file in the program.

Let us take a look at a few more opening modes used in the C programs:

Opening Modes of C in Standard I/O of a Program

Mode in Program	Meaning of Mode	When the file doesn't exist
R	Open a file for reading the content.	In case the file doesn't exist in the location, then fopen() will return NULL.
Rb	Open a file for reading the content in binary mode.	In case the file doesn't exist in the location, then fopen() will return NULL.
W	Open a file for writing the content.	In case the file exists, its contents are overwritten. In case the file doesn't exist in the location, then it will create a new file.
Wb	Open a file for writing the content in binary mode.	In case the file exists, then its contents will get overwritten. In case the file doesn't exist in the location, then it will create a new file.
A	Open a file for appending the content. Meaning, the data of the program is added to the file's end in a program.	In case the file doesn't exist in the location, then it will create a new file.
Ab	Open a file for appending the content in binary mode. Meaning, the data of the program is added	In case the file doesn't exist in the location, then it will create a new file.

	to the file's end in a program in a binary mode.	
r+	Open a file for both writing and reading the content.	In case the file doesn't exist in the location, then fopen() will return NULL.
rb+	Open a file for both writing and reading the content in binary mode.	In case the file doesn't exist in the location, then fopen() will return NULL.
w+	Open a file for both writing and reading.	In case the file exists, its contents are overwritten. In case the file doesn't exist in the location, then it will create a new file.
wb+	Open a file for both writing and reading the content in binary mode.	In case the file exists, its contents are overwritten. In case the file doesn't exist in the location, then it will create a new file.
a+	Open a file for both appending and reading the content.	In case the file doesn't exist in the location, then it will create a new file.
ab+	Open a file for both appending and reading the content in binary mode.	In case the file doesn't exist in the location, then it will create a new file.

How do we close a file?

Once we write/read a file in a program, we need to close it (for both binary and text files). To close a file, we utilise the fclose() function in a program.

Here is how the program would look like:

fclose(fptr);

In this case, the fptr refers to the file pointer that is associated with that file that needs to be closed in a program.

How do we read and write the data to the text file?

We utilise the fscanf() and fprintf() to write and read the data to the text file. These are basically the file versions of the scanf() and printf(). But there is a major difference, i.e., both fscanf() and fprintf() expect a pointer pointing towards the structure FILE in the program.

Example #1: Writing Data to the Text File in a Program

```
#include <stdio.h>

#include <stdlib.h>

int main()

{

int val;

FILE *fptr;

// if you are using Linux or MacOS, then you must use appropriate locations

fptr = fopen (“C:\\currentprogram.txt”,”w”);

if(fptr == NULL)
```

```
{

printf("File type invalid!");

exit(1);

}

printf("Please enter the val: ");

scanf("%d",&val);

fprintf(fptr,"%d",val);

fclose(fptr);

return 0;

}
```

The program mentioned here would take the number given by then store it in the currentprogram.txt file.

Once we compile this program and run it on the system, we will be able to witness a currentprogram.txt text file created in the C drive of our computer! Also, when we open this file, then we will be able to see what integer we entered as an input while coding.

Example #2: Reading Information from the Text File in a Program

```
#include <stdio.h>

#include <stdlib.h>

int main()
```

```
{

int val;

FILE *fptr;

// The program will exit in case the file pointer fptr returns NULL.

if ((fptr = fopen("C:\\currentprogram.txt","r")) == NULL){

printf("Visible error detected. Cannot open the file!");

exit(1);

}

fscanf(fptr,"%d", &val);

printf("The value of the integer n is=%d", val);

fclose(fptr);

return 0;

}
```

The program given here would read the integer that we have input in the currentprogram.txt file and then print the integer as an output on the computer screen.

In case you were successful in creating the file with the help of Example 1, then the running of the program in Example 2 will generate the integer that you have input.

You can also use other functions such as fputc(), fgetchar(), etc., in a similar manner in the program.

How do we Read and Write the Data to the Binary File in a Program?

We have to use the functions fwrite() and fread() for writing to and reading from a file present on a disk respectively, for the binary files.

When Writing to the Binary File

We utilise the function fwrite() for writing the data into a binary file in the program. This function would primarily take four arguments:

- the address of the data that would be written on the disk
- the size of this data
- total number of such types of data on the disk
- the pointers to those files where we would be writing

It would go something in this line:

fwrite (address_of_Data, size_of_Data, numbers_of_Data, pointerToFile);

When Reading from the Binary File

Just like the above-used function fwrite(), the function fread() also takes 4 arguments (lik above).

It would go something like this:

fread (address_of_Data, size_of_Data, numbers_of_Data, pointerToFile);

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.